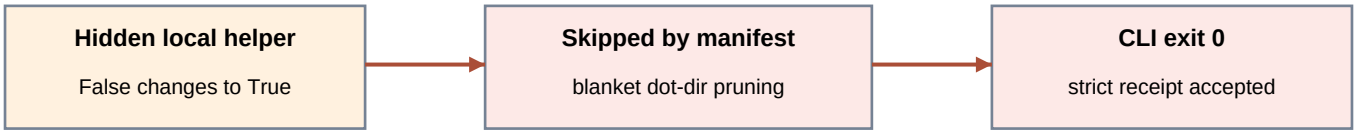
Independent review of Lean workflow repairs

TASK-20261018-695548 · J3-repair-workflow-boundaries · 2026-10-18

Own-joint verdict: breaks. The three original reproductions now reject within their ordinary input layouts. A remaining persistent imported-source change escapes the repaired CLI when Lake configures a hidden local source directory. The strict receipt validator also accepts that resulting receipt. This is a source-binding defect; Lean checks the changed proposition, and no proof of False or theorem refutation is asserted.

Real Lean checks the changed True proposition in both layouts.



TASK-20261018-695548 / frozen candidate only / no kernel refutation

Authority and exact candidate

The authority is the committed handoff and repair plan published at `d47080d59f4b5b54ffd29df46d8089c5665689c5` before this dispatch. All 32 manifest entries match both frozen `/workspace/lean-review-repairs` bytes and their bound Git blobs. Candidate revisions are CryptoLean `617b0a5433516796f9ea9d48109b8eb135f92dfc`, crypto `f1052053f01e7db23692e4ab4824f4df5a7bf29b`, cryptanalysis `93b5ce33fd5b3794e3fbc6b548986cb2ae72ccca`, autoresearcher `2951f9b77d96606820be80bf61ee0d539cfbb4dd`, and review `aaccd2f5d0a801fd73209e2a51b587efbda3ee5e`. Exact supporting runtime files were individually copied from that review revision; loaded repaired CLI/worker/receipt bytes agree with the manifest. See `artifacts/input-bindings.json`, `runtime-input-bindings.json`, and retained source snapshots.

I did not author these repairs. Only the permitted prior J3 report, its checks/attestation excerpts and two reproducer scripts were read. No J1/J2/J4 report or Coordinator synthesis was read. No subagent, commit, producer edit or scientific status change occurred. Every Lean command ran under this task's `/tmp/TASK-20261018-695548` tree. The original autoresearcher formal workspace was neither written nor used as a Lean working directory.

Observation: repaired original controls

Boundary	Fresh positive control	Fresh negative control	Result
Shared and both consumer gates	Configured, unimported valid module passes; two theorems audited	Unconfigured good or false source: exit 1, FAIL_build_source. Configured false source: exit 1, FAIL_audit/sorryAx	12/12 expected signatures
Gate ownership/exclusions	Outer project passes and lists only its own Root source	Separately discovered nested project with sorry fails its own audit	Declared boundary reproduced
CLI input binding	Ordinary valid named theorem: exit 0	Imported False before mutation: exit 1; False→True after version query: exit 3	Original failure repaired
Persistent input-set/software changes	Exact theorem checked by real worker	Add source; delete imported source after successful real audit; change target, audit, lakefile or scratch verifier: exit 3	Tested changes invalidate evidence
Strict receipts and active summary wrapper	Actual CLI receipt with nonempty project and exact-name audit: accepted, wrapper exit 0	Fifteen isolated identity/status/boolean/forbidden/log/version/input attacks and a stale helper reject; original combined forgery makes wrapper exit 1	Original contradictory acceptance repaired

Raw commands, stdout/stderr, build/audit logs and before/after source copies are in `artifacts/controls/`, `gate-receipts/`, and `control-sources/`. Every case is enumerated in `checks.json`; `controls-results.json` preserves the original attempts, and `followup-results.json` preserves the discriminating follow-ups. The controls use

actual pinned Lean 4.35.0 for gate projects and Lean 4.22.0/Lake 5.0.0 for the dependency-free CLI projects, not fake Lean output or test-suite assertions.

Breaking artifact: configured hidden local source

At repaired `orchestration/formal/receipts.py:27`, `workspace_inputs` removes every directory whose name starts with a dot. Its docstring describes dependency/build exclusions, while the skill promises complete local source/configuration coverage. Lake supports a normal local library with a hidden source directory:

```
[[lean_lib]]
name = "Helper"
srcDir = ".hidden"
```

`formal/.hidden/Helper.lean` defines `CryptoResearch.Control.P : Prop := False`. The unchanged target `CryptoResearch/Test.lean` imports Helper and states `theorem good : P := by trivial`. The frozen CLI returns exit **1** on those initial bytes. A wrapper around the real `lake env lean --version` subprocess changes only the helper's P from False to True immediately after the query. All later build and axiom checks still run real Lean.

The repaired CLI now returns exit **0**, `machine_verified`, with no infrastructure failure. The target's before/after SHA-256 is identical. The complete recorded `workspace_inputs` omits `.hidden/Helper.lean`; the strict `receipt_errors` returns **[]**. The matching hidden-layout positive also returns 0, so the result is not explained by an unsupported build or failed control. This helper is a configured local source, not a dependency binary or transient restored edit. The mutation remains present after verification.

The load-bearing artifacts are `artifacts/hidden-counterexample.json` and `artifacts/controls/followup-hidden-{positive,false-before,loadbearing-mutated}/`. Each contains source snapshots and raw receipts. The unchanged target has SHA-256 `420c3f280e415d8438f56afd1b4d6c9172bbd5cfe7b963d2b08808249df52a4f`. Lean proves the changed True statement; the evidence cannot be bound to the original complete local input set.

Comparison, fidelity and limits

The adapted review-evidence/formal skills correctly require fresh exact-name checks, independent semantic fidelity, explicit source inventory, and separate treatment of consistency, run authentication and mathematical validity. The active Node summary delegates to `orchestration.formal.receipts`, propagates failure and states its authentication/fidelity limit. Those claims are supported by the exercised commands. Its complete-local-input promise is too broad for the blanket hidden-directory pruning, so workflow fidelity is partial/mismatch and review outcome is needs_revision for this candidate.

The validator also conservatively rejects a successful exact-theorem receipt whose project audit has zero theorems in its selected namespace. My first `Control.good` positive reproduced that rejection; a new `CryptoResearch.Control.good` receipt with a nonempty project audit passed. This is false rejection, not acceptance of a bad proof. The Coordinator separately reported a scoped zero-axiom audit-format rejection; I received that message but did not read the serial-run artifacts or count them as independent evidence. The validator recognizes only its main-workspace nonempty audit string, so alternative valid project-audit formats need a new expressly supported contract.

One original deletion hook did not fire because the worker's default runner retained the original subprocess function. Its exit 0 is preserved and excluded from deletion evidence. The follow-up invokes the original real worker, confirms successful machine checks, then deletes the imported helper; it returns exit 3. Initial receipt/summary positive failures are also preserved, with their zero-count reason. The first setup tried a repository-documented Python path absent in this runtime, then used the handoff venv. A git-archive setup stalled and was interrupted; exact individual blobs supplied the runtime instead. These failed attempts were not rewritten.

No full historical rebuild, live CI, proof generation/MathCode, installed console command, external lean4checker fresh check or dependency-binary authentication was performed. Pre/post hashes do not establish absence of transient edits. Actual source-read paths and depth, including hash-only bindings and truncated reads, are in `artifacts/sources-read-details.json` and the attestation. No mathematical or performance canonical graph changes: this is a workflow boundary review. The diagram/PDF describe only the tested instrument behavior.

Next discriminating correction and reproduction

Bind hidden local proof/configuration sources, or reject unsupported local-source layouts before checking them; reserve exclusions for expressly declared infrastructure. A new committed candidate must pass the ordinary controls and the hidden positive, reject the original hidden False baseline, and invalidate the persistent False→True helper mutation with exit 3 and no

successful verification object. Keep the current failed candidate and logs immutable. Receipt format support should similarly receive a new positive/negative control rather than accepting arbitrary strings.

The retained scripts are `artifacts/bootstrap.py` (failed setup preserved), `bootstrap-v2.py` (exact runtime setup), `review_controls.py`, and `followup_controls.py`. Run the successful setup and the two control scripts in that order with `/tmp/lean-review-venv/bin/python`, fresh scratch/output names, and `PATH=/workspace/.elan/bin:$PATH ELAN_HOME=/workspace/.elan XDG_CACHE_HOME=/workspace/.cache PYTHONDONTWRITEBYTECODE=1`. They declare every source and command; existing output paths intentionally refuse overwrite. Archival publication belongs to TASK-20261018-95bdb1.

Full top-level `review_attestation` wrapper: review_attestation.yaml. It assigns only J3-repair-workflow-boundaries and records **breaks**; this is not a whole-claim approval vote.